

SAgrIoT — presentation script

4 September 2026 · RPTU Kaiserslautern-Landau · 15 slides · target 14–15 minutes

Pace: slide number ≈ minute. After slide 5 be at ~5 min, after 8 at ~8, after 11 at ~11. Two minutes behind at slide 8? Drop slide 8 — the table on slide 7 already proves that point.

Say these word for word — everything else can be improvised

- 1 A flat line cannot cross a threshold.
 - 2 The rules decide. The forecast only moves the moment forward.
 - 3 Accuracy does not tell you whether it will warn in time.
 - 4 Generality here does not mean a bigger training set. It means not needing one.
 - 5 Energy is not the constraint on a gateway. Memory is.
 - 6 I have not ported it. This is a calculation, not a demonstration.
 - 7 None of these were reported to me. I went looking.
-

1 · Cover — 1:00

Good afternoon, everyone. I'm Zehra, a computer engineering student at Gebze Technical University. I have been here since [MONTH], working on the decision layer of SAgrIoT.

My question was simple. If we put sensors in a greenhouse, can we tell a grower what to do — and tell them early enough to act?

Last time I showed you a system running on simulated soil. Since then I built a real one. It has been logging for twelve days without a break.

And it made me ask something I did not expect to be interesting: **which forecasting model do we actually need?**

The answer is: much less than I assumed. I will show you how I measured that, and also what I got wrong.

pause before moving on

2 · The system — one rule engine, used twice — 0:55

This is the decision layer. Sensors are read every thirty seconds. The readings go into a rule engine, and the engine produces a recommendation for right now.

The forecast is the second path. It takes the last twenty-four hours, predicts the next three, and sends those predicted rows into the **same** rules.

So this is not two systems. It is one rule engine, used twice — once on the present, once on the future.

That is the design choice that matters. **The rules decide. The forecast only moves the moment forward.** If the forecast is wrong, the advice comes early or late. It never becomes different advice.

3 · The rules, and where they come from — 1:05

At the last meeting I was asked to support the rules with published evidence. So I wrote a separate document. One chapter per rule, twenty-five pages, sixteen peer-reviewed sources. I sent it to Christian on Tuesday.

Every chapter has the same four sections. The third one is the one I want to point at: *honest scope of these references* — what the sources do **not** support.

The rules are not equally well evidenced. Rule 5 rests on a single source. I say that in its chapter instead of hiding it.

Writing those sections also changed the code. My tomato root-zone limit was thirty-five degrees. I went looking for a source. There wasn't one. It is now thirty.

slow down on the last sentence

4 · Where I was last time — 0:45

This is where I was in [MONTH]. The hardware was not connected. So I took real greenhouse air data and simulated the soil on top of it. Then I trained a model and tested it on a random split.

And I reported these numbers. Three percent error. R-squared 0.68. Ninety-one percent accuracy at predicting the drought threshold.

That was honest at the time. The pipeline worked. But every one of those numbers rests on something I could not check yet.

pause

5 • What changed — 1:25

Five things changed.

The soil is measured now, not simulated. Twelve days, thirty-four thousand readings, and the largest gap in the whole record is a hundred and forty-three seconds.

Nothing is trained any more, and I dropped the trained model on purpose. A trained model needs history from the site where it will run — and a greenhouse instrumented last week has none. So every method I compare now works from the first day, with no local data.

That is also my answer to making the work more general. **Generality here does not mean a bigger training set. It means not needing one.**

I have two environments instead of one. And seven rules instead of six.

But the row that matters is the last one. I stopped asking how accurate the forecast is. I started asking whether it warns in time.

Those are not the same question. **Accuracy does not tell you whether it will warn in time.** That is what the next few slides are about.

6 • How I measure it — 0:55

Briefly, how I measure — the next slide depends on it.

I slide a window along the record. Twenty-four hours of history, forecast three hours ahead, move forward one hour, repeat.

Then I score every forecast twice. At the signal level: how close was it. At the decision level: did it warn before the crossing, was it right when it warned, and how wrong was the timing.

One rule matters here. I only count windows that start on the safe side of the threshold. If the soil is already too dry, predicting that it is too dry is not an early warning.

7 • Accuracy and usefulness point in opposite directions — 1:20

This is the result I did not expect.

Start with persistence — the simplest method there is. Hold the last value. On several channels it has the **lowest error of anything I tested**. And its recall is zero. In every channel, in every dataset, across up to two hundred and thirty crossings. It never warned about anything.

The reason is not subtle. **A flat line cannot cross a threshold.** The most accurate method is structurally unable to warn.

Now look at the table. Soil moisture in the commercial greenhouse, a hundred and eight crossings. Seasonal naive has the worst accuracy on the slide — a skill of minus 0.43. And it catches eighty percent of the crossings.

So here, the worst forecast is the best detector. **Accuracy does not tell you whether it will warn in time.**

If I had chosen a method by error — which is what I did last time — I would have chosen one that cannot do the job.

8 • Accuracy does not predict usefulness — 0:50

The whole comparison in one picture. Eighty-four points: every method, every channel, both environments.

The green dot is persistence. All fifteen cases land on that one point. Zero on both axes.

The red points are seasonal naive. Most sit on the left — worse than doing nothing. Several sit near the top — they caught most of the crossings.

And the grey cloud has no shape. Across all eighty-four points, the correlation between accuracy and detection is 0.19. Almost nothing.

Accuracy does not tell you whether it will warn in time. That is the same sentence as two slides ago, now measured across everything I tested.

9 • Method choice does not transfer between sites — 1:10

Three channels. Three methods. Every one reverses between the two environments.

Soil moisture first. My physical model catches eighty-three percent of crossings in my pot, and between sixteen and twenty-six percent in the commercial greenhouse. The model encodes evaporation. That is what governs my pot. In the greenhouse an irrigation controller refills the soil on a schedule — evaporation is not the process running there.

Light and CO₂ go the other way. Repeating yesterday is one of the worst methods in my office and one of the best in the greenhouse.

The pattern is the same underneath. In the greenhouse these signals are decisions made by a machine. In my office they are weather, and people opening doors.

So you cannot pick a method once and ship it. It has to be measured per channel, at the site.

10 • What it costs to run — 1:15

If you have to measure it at every site, then what it costs to run matters as much as how well it works.

All of this is measured on the actual device — a Raspberry Pi 5 — each method in its own process, so the memory belongs to that method.

The spread is very large. The physical model: nine hundredths of a millisecond, no measurable memory. The small pretrained model: a hundred and twenty-two milliseconds, five hundred and thirteen megabytes.

Now the second block. A full cycle takes four hundred and fifty milliseconds, once every ten minutes. That is a duty cycle below one tenth of one percent. The device is idle ninety-nine point nine percent of the time.

So **energy is not the constraint on a gateway. Memory is** — the model holds that memory whether it computes or not.

One thing surprised me. Reading the log costs three hundred and fifty milliseconds. Four times more than forecasting everything. The dominant cost is not the model. It is the file.

11 • The irrigation decision fits on a microcontroller — 1:15

Now put those two slides together.

For the irrigation decision, the physical model matched the pretrained model's detection — both caught five crossings out of six. And it found the crossing time more than twice as precisely: eleven minutes of error against twenty-four and thirty-six.

So I asked what that model actually needs. A twenty-four hour buffer of soil moisture and VPD — about two and a half kilobytes. One two-by-two least squares. One exponential.

An ESP32 has five hundred and twenty kilobytes and costs a few euros. The model would use half a percent of it. The pretrained model holds five hundred and thirteen megabytes.

I want to be clear. **I have not ported it. This is a calculation, not a demonstration.**

But if it holds, it changes what a node costs. And in this project, cost per node decides whether anything gets deployed at all.

if asked — the cost measurement is hardware and it is solid. The detection comparison behind it rests on two crossings, and that is on the limitations slide.

12 • The early warning is probabilistic now — 1:10

One thing I added since the last meeting. The forecast is no longer a single line. Every channel is forecast five times, once per quantile, and all five go through the same rules. The share that fires becomes a probability.

The interesting part is that the same machinery behaves very differently on two rules.

Lighting is triggered by the clock. All five scenarios agree, every time, with zero spread. Here is one evening counting down from a hundred minutes to five without a single disagreement.

Water stress is triggered by physics. In sixty-nine warnings it was never once unanimous. Usually one or two scenarios out of five.

So the system is certain about the clock and unsure about the weather. That is exactly right. It separates what it knows from what it is guessing.

One caveat: this is a share of scenarios. It is not a calibrated probability, and I do not claim it is.

13 · I audited my own system — 1:20

I spent a day this week checking my own system instead of adding to it. Four things came out.

First: two of my six rules were active in **every one** of five hundred and sixty-seven cycles. Both are measurement artefacts, not plant conditions. My conductivity probe and my thresholds are on different scales. And light never passes the day threshold in my office, so daytime temperature is judged against night limits.

Second — and this is the one I would keep. A leaf covered my light sensor for six days. I found it afterwards in the night-time baseline. At night a dark room has a known value, so any step in it must be physical. The same reference showed my CO₂ baseline drifting upward over eight nights.

My stale-sensor check catches a frozen reading. It does not catch a slow drift or a partial obstruction. The quiet hours do.

Third: my field-capacity measurement worked once in three attempts. Fourth: one event was counted six times, because the signal oscillated around the limit.

None of these were reported to me. I went looking.

14 · What this does not show — 0:55

I want to state the limits myself rather than wait for them.

One pot, one room, twelve days. The plant is a chrysanthemum and the thresholds are tomato's — I use them as fixed reference levels, not as advice for that plant.

On the testbed my headline numbers rest on **two** physical crossings. Six windows, but they overlap. The statistical weight in this work is in the reference greenhouse, where each channel has between thirty-three and seventy-seven crossings.

Rule 3 I could not evaluate on the testbed at all.

And the two that matter most. There is no field deployment — nobody has ever acted on an advice this system produced. And the microcontroller number is a calculation, not a port.

do not rush this slide and do not apologise

15 · What's next — 1:05

Three things next. Port the irrigation path to a microcontroller and measure it, because right now that is arithmetic. Field deployment and multi-site validation. And more events — the seventh crossing is due in the next two days.

I would like to continue this as my graduation project at Gebze Technical University.

I am also writing a full report — the method, the results and the limitations in detail. It is not finished; I will send it before I leave on the sixteenth. I am writing it so whoever picks this up next has the reasoning, not only the code.

Thank you to Christian and the Microelectronic Systems Design group for having me, and to Professor Akgül for making it possible. And to [NAMES] — it has been good to work alongside you. Safe travels.

Before Friday

[MONTH] on slides 1 and 4 · [NAMES] on slide 15 · photos of the sensor setup near slide 4–5 · numbers that change after the new benchmark: slides **5, 6, 9, 11, 12, 14** — the Wageningen figures on slide 7 do not change.

How to practise

- 1 Read it aloud once, start to finish, without stopping. Time it.
- 2 Underline every sentence you stumble on and **rewrite it in your own words**. A sentence you can say beats a sentence that is well written.
- 3 Close the script. Present from the slides only.

Likely questions

How many events is that really? Two physical crossings on the testbed; thirty-three to seventy-seven per channel in the greenhouse.

Why not just train a better model? A new site has no history. Every method here works from day one.

Does it generalise? Method choice does not, and that is a result rather than a gap. The framework does.

Have you run it on a microcontroller? No. The cost measurement is hardware and it is solid; the port is the next step.

What happened to the RandomForest? It was dropped on purpose, not abandoned. A trained model cannot be validated on a site that has no data.